

Presentación: DOOM AUV Simulation — Simulador de AUV con Arquitectura ROS2

"Documento generado a partir del análisis exhaustivo de todo el código fuente del proyecto doom_aucv_sim."

PASO 1: Análisis Arquitectónico y Estructural

1.1 Visión General

DOOM AUV Sim es un simulador completo de un **Vehículo Autónomo Submarino (AUV)** construido sobre ROS2 y Python. El sistema simula la operación de un AUV tipo Girona 500 en un entorno submarino procedural, incluyendo:

- Renderizado en primera persona estilo *DOOM* (raycasting 2.5D) con Pygame
- Motor de física subacuática con inercia, drag hidráulico y batería
- Canal de comunicación acústica con latencia, pérdida de paquetes y modelo SNR
- Misiones autónomas con exploración por waypoints, detección de objetivos y recolección con brazo robótico
- Arbitraje de comandos entre control manual y autónomo (arquitectura inspirada en subsunción)
- Exportación del mapa explorado a PCD, PNG y OBJ 3D
- Visualización paralela en RViz2 con delay realista

1.2 Topología ROS2

Nodos (6 + RViz2 + Teleop)

Nodo	Archivo	Rol
doom_sim_node	main_sim_node.py	Simulador principal: física, renderizado, publicación de sensores
teleop_node	teleop_node.py	Interfaz de operador de superficie (terminal con dashboard)
acoustic_channel_node	acoustic_channel_node.py	Canal bidireccional con latencia y pérdida de paquetes
arbitrator_node	arbitrator_node.py	Multiplexor de comandos Manual/Auto → Thruster

mission_node	mission_node.py	Lógica de autonomía onboard: exploración, aproximación, alineación, recolección
viz_bridge_node	viz_bridge_node.py	Puente con delay realista para RViz2 (TF + PointCloud + Sonar)
rviz2	(externo)	Visualización 3D estándar ROS2

Tópicos Principales (~20 tópicos)

```
graph LR
    subgraph Topside ["Operador de Superficie"]
        TN[teleop_node]
    end

    subgraph Acoustic ["Canal Acústico"]
        AC[acoustic_channel_node]
    end

    subgraph Onboard ["Electrónica Onboard"]
        ARB[arbitrator_node]
        MN[mission_node]
    end

    subgraph Simulator ["Motor de Simulación"]
        SIM[doom_sim_node]
    end

    subgraph Viz ["Visualización"]
        VB[viz_bridge_node]
        RV[RViz2]
    end

    TN -->|"/comm/topside_tx"| AC
    AC -->|"/comm/auv_rx"| ARB
    AC -->|"/comm/auv_rx"| MN

    MN -->|"/comm/auv_tx"| AC
    AC -->|"/comm/topside_rx"| TN

    MN -->|"/cmd_vel/autonomous"| ARB
    MN -->|"/auv/control_mode"| ARB
    MN -->|"/auv/arm_cmd"| SIM

    ARB -->|"/auv/thruster_cmd"| SIM

    SIM -->|"/sim/state"| VB
    SIM -->|"/sim/sonar_scan"| VB
    SIM -->|"/sim/point_cloud"| VB
    SIM -->|"/sim/map_cloud"| VB

    VB -->|"/auv/state"| MN
    VB -->|"/auv/sonar_scan"| MN
    VB -->|TF: map→base_link| RV
    VB -->|"/auv/map_cloud"| RV

    SIM -->|"/auv/detected_targets"| MN
    SIM -->|"/auv/arm_state"| MN
    SIM -->|"/auv/battery"| MN
    SIM -->|"/auv/mission_event"| MN
```

Servicios y Acciones

El proyecto **no utiliza Servicios ni Acciones ROS2**; toda la comunicación es asíncrona basada en **Tópicos Pub/Sub**, lo cual es coherente con la naturaleza de tiempo real del control robótico y la

simulación de latencia acústica.

Parámetros Clave (Constantes Hardcoded)

Parámetro	Valor	Archivo	Significado
TILE_SIZE	32	physics.py	Escala del mapa: 1 celda = 32 píxeles ≈ 1 metro
MAP_SIZE	100×100	map_manager.py	Área del mapa: 100×100 metros
WATER_DRAG_LINEAR	0.92	physics.py	Factor de inercia lineal (retiene 92%/tick)
WATER_DRAG_ANGULAR	0.85	physics.py	Factor de inercia angular
FOV	π/3 (60°)	renderer.py	Campo de visión del sonar/cámara
CASTED_RAYS	120	renderer.py	Resolución del raycasting
ACOUSTIC_RANGE	3000 m	communication_logic.py	Alcance del módem acústico
FREQUENCY_KHZ	25 kHz	communication_logic.py	Frecuencia acústica operativa

1.3 Módulos Auxiliares (No-Nodos)

Módulo	Responsabilidad
physics.py	Motor de física: actualización de posición, colisión con paredes/obstáculos, batería, SNR
renderer.py	Renderizado DOOM: raycasting, sprites, HUD, sonar minimap, brazo robótico
map_manager.py	Generación procedural del mapa, obstáculos dinámicos/estáticos, targets
communication_logic.py	Modelo físico de comunicación: Thorp absorption, SNR acústico/óptico, Doppler, scintillation

1.4 Flujo de Datos (Ciclo de Vida Completo)

```
1. LAUNCH (doom_sim.launch.py)
■■■ doom_sim_node → Inicializa Pygame, PhysicsEngine, MapManager, Renderer
■■■ acoustic_channel_node → Espera mensajes bidireccionales
■■■ viz_bridge_node → Buffer con delay aleatorio (0.1-0.3s)
■■■ arbitrator_node → Modo MANUAL por defecto
■■■ mission_node → Modo MANUAL, waypoints generados
■■■ rviz2 → Carga configuración .rviz

2. MANUAL (Teleoperación)
Operador → [tecla W] → teleop_node →(JSON)→ /comm/topside_tx
→ acoustic_channel (delay + 5% loss) → /comm/auv_rx
→ arbitrator_node (MANUAL mode) → /auv/thruster_cmd
→ doom_sim_node → physics.update() → render → /sim/state
→ viz_bridge_node (delay) → /auv/state → mission_node (telemetry)
→ /comm/auv_tx → acoustic_channel → /comm/topside_rx → dashboard

3. AUTO (Misión Autónoma) – Máquina de Estados
EXPLORE → waypoints lawnmower → detecta target visible
→ WAIT_CONFIRM → operador confirma via acústico
```

```
→ APPROACH → navegación al target (control proporcional)
→ ALIGN → visual servoing del brazo (pantalla) + rotación gripper
→ GRIP → cierra + verifica → target_collected
→ RECOVER (2s) → EXPLORE (siguiente waypoint)
```

4. SEGURIDAD (Subsunción – prioridad máxima)
- EMERGENCY_REVERSE: obstáculo < 1.5m → retroceso puro 2s
 - AVOID_CRITICAL: obstáculo < 3.0m → retrocede + gira 3s
 - BATTERY_FAILSAFE: batería < 10% → fuerza ascenso emergencia
 - SURFACING: misión completa (5 targets) → guarda mapa + asciende

PASO 2: Estructura de la Presentación (8–10 Diapositivas, ~10 min)

Diapositiva 1: Portada (30 segundos)

Título: *Simulación de AUV con Arquitectura ROS2 — Motor de Renderizado DOOM*

Contenido Visual:

- Logo de la universidad
- Screenshot de la interfaz Pygame (ventana principal con HUD)
- Nombre del autor, asignatura, fecha

Puntos Clave:

- Proyecto: Simulador de Vehículo Autónomo Submarino
 - Stack: ROS2 Humble + Python + Pygame
 - Inspiración visual: raycasting al estilo *DOOM* (1993)
-

Diapositiva 2: Motivación y Contexto (1 min)

Título: *¿Por qué simular un AUV con visualización en primera persona?*

Contenido Visual:

- Foto real del Girona 500 AUV
- Comparación lado a lado: vista DOOM vs. vista RViz2

Puntos Clave:

- Los AUV operan sin comunicación GPS; la supervisión es limitada
 - La comunicación acústica submarina tiene alta latencia (~1s) y pérdida de paquetes (~5%)
 - Un simulador en primera persona permite empatizar con las limitaciones sensoriales del robot
 - RViz2 complementa con la visión "omnisciente" del mapa 3D
-

Diapositiva 3: Arquitectura General del Sistema (1.5 min)

Título: *Topología ROS2: 6 Nodos, ~20 Tópicos, 0 Servicios*

Contenido Visual:

- **Diagrama Mermaid** (o equivalente `rqt_graph`) del flujo de nodos y tópicos
- Agrupar por: Topside, Canal Acústico, Onboard, Simulador, Visualización

Puntos Clave:

- Pub/Sub puro: coherente con sistemas de tiempo real y latencia variable
 - Separación clara: Simulación (`/sim/*`) vs. Percepción (`/auv/*`)
 - El `viz_bridge_node` introduce delay realista entre simulador y RViz
 - El `acoustic_channel_node` es el cuello de botella deliberado de comunicación
-

Diapositiva 4: Motor de Simulación — Física y Renderizado (1.5 min)

Título: *doom_sim_node: Pygame + Raycasting + Física Subacuática*

Contenido Visual:

- Screenshot de la ventana Pygame con anotaciones (HUD, sonar minimap, brazo, muro 3D)
- Fragmento de código del `game_loop` (5–8 líneas)

Puntos Clave:

- **Raycasting** a 120 rayos con corrección de fisheye → paredes en pseudo-3D
 - **Fog of War**: el sonar solo revela celdas visitadas (mapa 100×100)
 - **Física**: velocidad acumulativa + drag hidráulico (`WATER_DRAG = 0.92`) = inercia realista
 - Publicaciones: `PoseStamped`, `LaserScan`, `PointCloud2`, `Marker`, `Float32` (batería)
-

Diapositiva 5: Canal de Comunicación Acústica (1 min)

Título: *Simulación de Comunicación Submarina: Latencia, SNR y Pérdida de Paquetes*

Contenido Visual:

- Diagrama del flujo Topside ↔ Canal ↔ AUV
- Fórmula de Thorp ($\alpha = 0.11f^2 / (1+f^2) + \dots$)
- Gráfico conceptual de SNR vs. distancia

Puntos Clave:

- Modelo de Thorp para absorción acústica a 25 kHz
 - Delay = distancia/1500 m/s + ruido aleatorio (0.01–0.05s)
 - 5% de pérdida de paquetes estocástica
 - Modelo óptico exponencial para enlace laser azul (rango 100m)
 - Efecto de escintilación: cortes aleatorios del 5%
-

Diapositiva 6: Arquitectura de Control — Arbitraje y Subsunción (1.5 min)

Título: *Control Híbrido: Teleoperación + Autonomía con Arbitraje de Prioridad*

Contenido Visual:

- Diagrama de la arquitectura de subsunción (capas de prioridad)
- Tabla de prioridades: Emergency Reverse > Avoid Critical > Battery Failsafe > Mission States

Puntos Clave:

- **arbitrator_node**: multiplexor Manual/Auto → solo una fuente controla los thrusters
 - **Subsunción** en **mission_node**: las capas de seguridad **anulan** el control manual
 - **< 1.5m** → EMERGENCY_REVERSE (fuerza AUTO, retroceso 2s)
 - **< 3.0m** → AVOID_CRITICAL (retrocede + gira hacia más espacio libre)
 - **< 10%** batería → SURFACING forzado
 - Todo el control manual pasa por el canal acústico (sin atajos directos)
-

Diapositiva 7: Misión Autónoma — Máquina de Estados (1.5 min)

Título: *Autonomía Onboard: Exploración, Detección y Recolección*

Contenido Visual:

- Diagrama de estados (EXPLORE → WAIT_CONFIRM → APPROACH → ALIGN → GRIP → RECOVER)
- Screenshot de la interfaz de teleop mostrando "WAITING CONFIRMATION: YES"

Puntos Clave:

- **EXPLORE:** patrón *lawnmower* con waypoints predefinidos (grid 15–95, paso 20)
 - **WAIT_CONFIRM:** el operador remoto debe confirmar/rechazar el target vía acústico
 - **APPROACH:** navegación proporcional (P-control en yaw + velocidad adaptativa)
 - **ALIGN:** *visual servoing* — alinea brazo a coordenadas de pantalla del target + rotación del gripper
 - **GRIP:** cierre + verificación (tolerancia: distancia < 1.5m, ángulo < 30°, pixel < 50px)
 - Tras 5 recolecciones → guarda mapa (PCD + PNG + OBJ) → ascenso
-

Diapositiva 8: Entorno y Mapa Procedural (45 seg)

Título: *Mundo Submarino Procedural: Obstáculos Dinámicos y Estáticos*

Contenido Visual:

- Screenshot del sonar minimap (esquina superior derecha)
- Tabla de tipos de obstáculo

Puntos Clave:

- Mapa 100×100 celdas con bordes sólidos + 50 paredes aleatorias
 - **Obstáculos dinámicos:** 10 barcos (0–10m), 10 criaturas (40–60m) con movimiento y rebote
 - **Obstáculos estáticos:** 15 arrecifes (17–37m), 15 minas (70–90m)
 - 5 targets aleatorios con profundidad (5–95m) y orientación (0–360°)
 - Fog of War: solo se muestra lo explorado por los rayos del sonar
-

Diapositiva 9: Visualización RViz2 y Exportación del Mapa (45 seg)

Título: Visualización 3D con RViz2 y Exportación Multi-Formato

Contenido Visual:

- Screenshot de RViz2 con PointCloud + TF + Marker
- Ejemplo de archivo PNG y OBJ exportados

Puntos Clave:

- `viz_bridge_node` reescribe timestamps para compatibilidad con TF
 - PointCloud2 con campo RGB empaquetado (paredes, obstáculos, targets)
 - Exportación al completar misión: **PCD** (nube de puntos), **PNG** (mapa 2D), **OBJ** (mesh 3D)
 - El modelo OBJ genera cubos 3D para paredes (0 a -100m) y obstáculos
-

Diapositiva 10: Conclusiones y Demo (~ 1 min)

Título: Conclusiones y Trabajo Futuro

Contenido Visual:

- Resumen de estadísticas (6 nodos, 20 tópicos, 10 ficheros Python, ~1800 LOC)
- Video corto o GIF de la simulación en acción (opcional)

Puntos Clave:

- Sistema completo de simulación AUV que integra física, percepción, comunicación y autonomía
 - Demuestra conceptos clave: latencia acústica, subsunción, visual servoing, fog of war
 - **Limitaciones:** no hay planificación de trayectorias (Nav2), targets con pose fija
 - **Trabajo futuro:** integración con Nav2, modelos de corriente marina, multi-AUV
-

PASO 3: Guion Narrativo (Talking Points)

Diapositiva 1 — Portada (~30s)

""Buenos días/tardes. Mi nombre es [nombre] y voy a presentar mi proyecto de simulación de un Vehículo Autónomo Submarino, o AUV, construido sobre ROS2. Lo que hace especial a este simulador es que utiliza un motor de renderizado inspirado en el clásico videojuego DOOM de

1993, con raycasting en primera persona, para representar el punto de vista limitado del robot. El sistema está desarrollado íntegramente en Python usando Pygame para la visualización y la infraestructura estándar de ROS2 para la comunicación entre componentes.""

Diapositiva 2 — Motivación (~1 min)

""¿Por qué es relevante simular un AUV de esta manera? Los vehículos submarinos autónomos operan en un entorno extremadamente difícil para las comunicaciones: no hay GPS bajo el agua, la comunicación acústica tiene latencias del orden de segundos y tasas de pérdida de paquetes significativas, y la visibilidad óptica se limita a pocas decenas de metros.

 Lo que este proyecto hace es obligar al operador humano a experimentar estas limitaciones de primera mano. La vista en perspectiva tipo DOOM simula lo que el robot 've' a través de su sonar frontal, mientras que la visualización en RViz2 proporciona una vista complementaria, más parecida a lo que un operador vería en una estación de control real, pero con el delay acústico incluido. De esta forma, el operador no puede hacer trampas: sus comandos llegan al AUV con retardo real y pueden perderse.""

Diapositiva 3 — Arquitectura General (~1.5 min)

""El sistema se compone de seis nodos ROS2 propios más RViz2. Toda la comunicación es Pub/Sub; no utilizamos Servicios ni Acciones de ROS2, lo cual es una decisión deliberada: en un sistema con latencia variable, las llamadas síncronas serían problemáticas.

 El nodo principal, `doom_sim_node`, es el motor de simulación: contiene la física, el renderizado y publica el ground truth en tópicos prefijados con `/sim/`. El `viz_bridge_node` toma estos datos y los republica en tópicos `/auv/` con un delay aleatorio de 100 a 300 milisegundos, simulando la latencia de los sensores reales.

 Por otro lado, tenemos el canal de comunicación acústica, implementado como un nodo independiente que introduce delay y pérdida de paquetes bidireccional entre el operador en superficie y la electrónica onboard del AUV. El `arbitrator_node` decide en cada instante si el robot atiende al control manual o al autónomo. Y el `mission_node` es el cerebro autónomo onboard que implementa la lógica de misión.

 Esta separación en nodos refleja una arquitectura real de AUV, donde la simulación es un componente sustituible y la lógica de control es independiente del hardware.""

Diapositiva 4 — Motor de Simulación (~1.5 min)

""El corazón del simulador es el `doom_sim_node`, que ejecuta un bucle principal a 20 Hz. En cada iteración se procesan tres fases: actualización de la física, renderizado y publicación de datos ROS2.

 El renderizado utiliza raycasting: se lanzan 120 rayos en un arco de 60 grados, y para cada rayo se detecta la primera pared. Con esa distancia se calcula la altura de la

columna que se dibuja en pantalla, aplicando corrección de fisheye. Además, a medida que los rayos exploran el mapa, se levanta el 'fog of war', un mecanismo clásico que oculta las zonas no visitadas.

 La física simula inercia subacuática: en lugar de asignar velocidad directamente, los comandos aplican una fuerza que se acumula, y en cada tick la velocidad se multiplica por un factor de drag de 0.92, simulando la resistencia del agua. Esto produce un movimiento suave y realista, donde el robot no se detiene instantáneamente.

 El nodo también publica la nube de puntos, el escaneo láser tipo sonar, el estado del brazo, las posiciones de los targets detectados, y el nivel de batería.""

Diapositiva 5 — Canal Acústico (~1 min)

""La comunicación submarina es uno de los aspectos más diferenciadores del proyecto. El nodo `acoustic_channel_node` implementa un canal bidireccional que simula las condiciones reales de un módem acústico.

 El modelo de propagación utiliza la fórmula de Thorp para calcular la absorción acústica a 25 kilohercios, una frecuencia típica en comunicaciones submarinas. El delay de transmisión se calcula como la distancia dividido por 1500 metros por segundo — la velocidad del sonido en el agua — más un componente aleatorio de procesamiento.

 Además, cada paquete tiene un 5% de probabilidad de perderse completamente, lo cual afecta tanto al control manual como a la telemetría de retorno. También se modelan efectos de escintilación — cortes de señal causados por ondas internas — y se calculan el SNR acústico y óptico que se muestran en el HUD del simulador.""

Diapositiva 6 — Arquitectura de Control (~1.5 min)

""El control del AUV sigue una arquitectura inspirada en la subsunción de Brooks. Hay dos fuentes de comandos: el operador de superficie enviando Twist messages a través del canal acústico, y el nodo de misión autónoma. El `arbitrator_node` actúa como multiplexor y solo deja pasar una de las dos fuentes hacia los thrusters del simulador.

 Lo más interesante es la jerarquía de seguridad implementada en el `mission_node`. Independientemente del modo activo — manual o autónomo — si el sonar detecta un obstáculo a menos de 3 metros, el sistema fuerza el modo autónomo y ejecuta una maniobra de evasión: retrocede y gira hacia el lado con más espacio libre. Si el obstáculo está a menos de 1.5 metros, primero ejecuta un retroceso de emergencia puro durante 2 segundos antes de la evasión rotacional.

 Del mismo modo, si la batería cae por debajo del 10%, el sistema fuerza un ascenso de emergencia independientemente de lo que el operador esté haciendo. Esta jerarquía garantiza la seguridad del vehículo por encima de cualquier objetivo de misión.""

Diapositiva 7 — Misión Autónoma (~1.5 min)

""La lógica de misión se implementa como una máquina de estados con seis estados principales. En modo EXPLORE, el AUV sigue un patrón lawnmower o de barrido — una rejilla de waypoints — navegando con un controlador proporcional en yaw y velocidad adaptativa.

 Cuando detecta un target visible, transiciona a WAIT_CONFIRM, donde se detiene y espera que el operador remoto confirme o rechace el objetivo. Esta confirmación viaja por el canal acústico, así que puede tardar y puede perderse.

 Una vez confirmado, entra en APPROACH: navegación proporcional hacia el target con velocidad decreciente. Al llegar a 1.5 metros y con el heading aproximadamente alineado, transiciona a ALIGN, que es la fase más sofisticada: utiliza visual servoing para alinear el brazo robótico con las coordenadas de pantalla del target. Simultáneamente, rota el gripper para que coincida con la orientación del target y avanza lentamente.

 Cuando la posición en pantalla, la rotación y la distancia 3D están dentro de tolerancia, cierra el gripper. Si la captura es exitosa — verificada por el renderer según proximidad y ángulo — el sistema emite un evento `target_collected`. Tras 5 recolecciones, guarda el mapa en tres formatos y asciende.""

Diapositiva 8 — Entorno Procedural (~45s)

""El entorno es un mapa procedural de 100 por 100 celdas con bordes sólidos y 50 obstáculos de pared aleatorios. Sobre este mapa se colocan obstáculos en cuatro capas de profundidad: barcos en superficie, arrecifes a media profundidad, criaturas marinas profundas y minas en el fondo. Los barcos y las criaturas se mueven dinámicamente, rebotando en los bordes del mapa.

 Los 5 targets de recolección se colocan en posiciones y profundidades aleatorias, con una orientación que el gripper debe igualar. Todo esto se descubre progresivamente gracias al sistema de fog of war: el minimap del sonar solo muestra lo que los rayos han explorado.""

Diapositiva 9 — Visualización y Exportación (~45s)

""Complementando la vista DOOM, el `viz_bridge_node` proporciona datos a RViz2 con un delay simulado. Esto incluye la transformada TF `map` → `base_link` para posicionar al robot, nubes de puntos con colores RGB empaquetados para paredes y obstáculos, y marcadores de visualización.

 Al completar la misión, el sistema exporta el mapa explorado en tres formatos: PCD para uso con herramientas de nube de puntos, PNG como imagen 2D coloreada por tipo de celda, y un archivo OBJ de Wavefront como mesh 3D, donde las paredes se representan como pilares de 100 metros de alto y los obstáculos como cubos a su profundidad real.""

Diapositiva 10 — Conclusiones (~1 min)

""En resumen, este proyecto demuestra cómo construir un simulador de AUV completo utilizando ROS2 como middleware de comunicación y un motor de renderizado clásico como interfaz de usuario. El sistema integra conceptos clave de robótica submarina: la limitación de las comunicaciones acústicas, la importancia de la autonomía onboard con mecanismos de seguridad de tipo subsunción, y técnicas de percepción como fog of war y visual servoing.

 Las principales limitaciones son la ausencia de un planificador de trayectorias completo como Nav2 — actualmente se usa navegación reactiva — y que los targets no cambian de posición. Como trabajo futuro, se podría integrar Nav2 para planificación global, añadir modelos de corrientes marinas que afecten a la navegación, o extender a una arquitectura multi-AUV cooperativa.

 Muchas gracias. Estoy disponible para preguntas.""

PASO 4: Preparación para Defensa (Q&A)

Pregunta 1: "¿Por qué se utiliza exclusivamente Pub/Sub en lugar de Servicios o Acciones ROS2 para la comunicación entre nodos?"

Respuesta:

La decisión de usar exclusivamente tópicos Pub/Sub responde a la naturaleza asíncrona y con latencia variable del sistema. En un escenario real de comunicación acústica submarina, el tiempo de ida y vuelta puede ser de varios segundos. Los Servicios ROS2 (**srv**) son síncronos y bloqueantes: el cliente espera la respuesta, lo cual provocaría bloqueos inaceptables en el bucle de control. Las Acciones (**action**) añaden complejidad para gestionar feedback y cancelación que, en este caso, se resuelve más elegantemente con la máquina de estados del **mission_node** y eventos publicados en tópicos. Además, la pérdida de paquetes simulada haría que los servicios fallaran con timeouts, mientras que los tópicos se limitan a "mejor esfuerzo": si un paquete se pierde, el siguiente llegará sin bloquear el sistema.

Pregunta 2: "¿Cómo garantiza el sistema la seguridad del AUV cuando la comunicación con el operador se pierde completamente?"

Respuesta:

El sistema implementa múltiples capas de seguridad, todas en el **mission_node** (onboard):

1. **Watchdog de comunicación** (línea 194): si no se recibe ningún mensaje de estado del simulador en 2 segundos, el sistema desactiva la autonomía y pasa a MANUAL como modo seguro. Esto protege contra desconexiones completas del simulador.

2. **Evasión de colisión reactiva** (líneas 220–263): opera en **todos los modos**, incluyendo manual. Si un obstáculo está dentro del umbral crítico, el sistema *anula* el control del operador, fuerza AUTO y ejecuta la maniobra de evasión. El operador no puede desactivar este comportamiento. Esto sigue el paradigma de subsunción donde las capas de seguridad tienen prioridad absoluta.

3. **Failsafe de batería** (líneas 203–215): si la batería baja del 10%, el sistema fuerza un ascenso de emergencia independientemente del modo activo.

4. **Surfacing por completitud**: una vez recogidos los 5 targets, el sistema guarda los datos y asciende automáticamente.

Estos mecanismos operan onboard, sin depender de la comunicación con superficie.

Pregunta 3: "¿Cuáles son las limitaciones del modelo de comunicación acústica implementado y cómo se podría mejorar?"

Respuesta:

El modelo actual tiene tres limitaciones principales:

1. **Delay determinístico**: el delay se calcula como `distancia/1500 + ruido_uniforme(0.01, 0.05)`, pero en la realidad el canal acústico presenta multipath (reflexiones), refracción por gradientes de temperatura (perfil de velocidad del sonido), y reverberación. Un modelo más realista utilizaría el modelo de Bellhop o trazado de rayos acústicos para calcular la estructura del canal.

2. **Pérdida de paquetes independiente**: el 5% de pérdida se aplica de forma **independiente** a cada paquete, pero en la realidad las pérdidas son **correlacionadas** — un mal canal permanece mal durante ventanas de tiempo. Se podría modelar con un canal de Markov de dos estados (Gilbert-Elliott).

3. **Ausencia de efectos en el dominio de la frecuencia**: el modelo incluye la absorción de Thorp y calcula el Doppler, pero no aplica realmente el efecto Doppler a los datos (no hay decodificación). En un sistema real, el shift Doppler causa errores de decodificación que aumentan con la velocidad del AUV. La función `calculate_doppler_shift` está implementada en `communication_logic.py` pero no se invoca activamente en el flujo principal.

Una mejora sería integrar una librería como DESERT (DEsign of underwater SEnsor neTworks) o usar datos reales de canal acústico para driving el modelo de simulación.